

Analyzing the Role of AI-Assisted Programming in Modern Software Development

IT 3003 - Advanced Programming Concepts

S.G.H.Lakthilina

17626

2023s20383

Table of Contents

1. Task 1	
1.1 Introduction AI Assisted Programming	3-4
1.2 Types	4-5
1.3 Advantages	5-6
1.4 Risk and limitation	6-7
2. Task 2	
2.1 Problem Description	8
2.2 Versions	
Version A	8-9
Version B	10
2.3 Outputs	11
2. .4 Comparison Between Version A and Version B	12-13
2.5 UML Diagrams	13-16
3. Task 3	
Critical Analysis	17
4. Task 4	
Conclusion	18
5. Reference	19
6. Github link	20

Task 01

1.1 Introduction AI Assisted Programming

AI-assisted programming is the use of Artificial Intelligence (AI) tools to help developers write, understand, test, and improve computer programs. Instead of writing every line of code manually, programmers can use AI to receive suggestions, generate code, find errors, explain complex code, and automate repetitive tasks. AI does not replace programmers, but it acts as a smart assistant that helps them work more efficiently and accurately.

Today, AI-assisted programming has become an important part of modern software development. Many popular tools, such as GitHub Copilot, ChatGPT, Amazon Q, and Codeium, use large language models that have been trained on large amounts of programming data. These tools understand programming languages and can provide useful recommendations while developers are writing code. For example, if a programmer writes a function name or a short comment describing what they want, the AI tool can suggest the rest of the code automatically. This helps reduce the time spent on routine coding tasks.

One of the main advantages of AI-assisted programming is that it increases productivity. Developers can complete coding tasks faster because AI provides code suggestions, identifies common mistakes, and recommends possible solutions. This allows programmers to spend more time on solving complex problems instead of writing repetitive code. AI is also useful for beginners because it can explain programming concepts, provide sample code, and help them understand error messages. As a result, students and new programmers can learn programming more effectively.

Another important feature of AI-assisted programming is debugging and testing. AI tools can detect syntax errors, identify possible bugs, and suggest improvements to make code cleaner and easier to maintain. Some tools can even generate unit tests automatically, helping developers ensure that their programs work correctly. This improves the overall quality and reliability of software while reducing the time required for testing.

Despite its benefits, AI-assisted programming also has some limitations. AI-generated code may not always be correct, secure, or efficient. Sometimes it produces incorrect logic or code that does not fully meet the user's requirements. Because of this, programmers must carefully review, test, and understand any code generated by AI before using it in a real project. Developers should not depend completely on AI, as human knowledge, critical thinking, and problem-solving skills are still essential for creating high-quality software.

Another challenge is that AI tools may generate code based on existing patterns from training data, which can sometimes include outdated practices or security vulnerabilities. There are also concerns about privacy and intellectual property when using AI tools with confidential source code. Therefore, organizations should use AI responsibly and follow appropriate security and ethical guidelines.

In conclusion, AI-assisted programming is transforming the way software is developed by making coding faster, easier, and more accessible. It supports programmers by generating code, detecting errors, explaining concepts, and improving productivity. However, AI should

be considered a supportive tool rather than a replacement for human developers. The best results are achieved when programmers combine their own knowledge, creativity, and critical thinking with the capabilities of AI. As AI technology continues to improve, it will become an even more valuable assistant in software development and programming education.

1.2 Types of AI coding tools

AI coding tools are software applications that use Artificial Intelligence (AI) to help developers during different stages of software development. These tools can generate code, identify errors, improve existing programs, create documentation, and automate repetitive tasks. They help developers save time, improve code quality, and increase productivity. Today, AI coding tools are widely used by students, software engineers, and organizations to make programming faster and more efficient.

As a student, I have found that AI coding tools are useful when learning new programming concepts and solving coding problems. They can explain errors, suggest better coding practices, and provide sample solutions. Some of the most popular AI coding tools include **GitHub Copilot**, **ChatGPT**, **Amazon Q Developer**, **Codeium**, and **Tabnine**. Although these tools are very helpful, developers should always understand and verify the generated code before using it.

The following are some of the main types of AI coding tools.

• **Code Generation Tools**

These tools automatically generate code based on user instructions, comments, or prompts. They help developers write functions, classes, or even complete programs more quickly. This reduces repetitive coding and increases productivity.

Examples: GitHub Copilot, ChatGPT, Amazon Q Developer.

• **Debugging Tools**

AI debugging tools help identify syntax errors, logical mistakes, and runtime issues. They also explain why an error occurred and suggest possible solutions, making it easier to fix bugs.

Examples: ChatGPT, GitHub Copilot, Visual Studio IntelliCode.

• **Code Refactoring Tools**

These tools improve the quality and readability of existing code without changing its functionality. They can rename variables, remove duplicate code, simplify complex functions, and organize the code according to best practices.

Examples: GitHub Copilot, JetBrains AI Assistant, Visual Studio IntelliCode.

• **Code Review Tools**

AI-powered code review tools examine source code to identify coding mistakes, security vulnerabilities, and performance issues. They also suggest improvements that help developers maintain high-quality software.

Examples: Amazon Q Developer, GitHub Copilot, DeepCode (Snyk Code).

- **Testing Tools**

AI testing tools automatically generate unit tests, create test cases, and identify sections of code that need additional testing. This helps improve software reliability while reducing manual testing effort.

Examples: Diffblue Cover, Testim, CodiumAI (Qodo).

- **Documentation Tools**

These tools automatically generate code comments, API documentation, and explanations for complex code. Good documentation makes software projects easier to understand and maintain.

Examples: ChatGPT, Mintlify AI, GitHub Copilot.

- **Code Completion Tools**

AI code completion tools provide real-time suggestions while developers are typing. They predict the next line of code, complete functions, and recommend commonly used programming patterns, helping developers code faster.

Examples: Tabnine, GitHub Copilot, Codeium.

- **Code Translation Tools**

These tools convert code from one programming language to another. They are useful when migrating applications or rewriting software using a different programming language.

Examples: ChatGPT, Amazon Q Developer, Google Gemini.

In summary, AI coding tools support developers in many areas of software development, from writing and debugging code to testing, documentation, and code improvement. Different tools are designed for different purposes, but they all help improve productivity and software quality. However, AI-generated code should always be reviewed and tested to ensure it is accurate, secure, and suitable for the project.

1.3 Advantages in software development

Artificial Intelligence (AI) has become an important technology in modern software development. It helps developers perform many programming tasks more efficiently by providing code suggestions, identifying errors, and automating repetitive work. AI-assisted programming improves productivity and allows developers to focus on solving complex problems instead of spending time on routine coding tasks. As AI technology continues to improve, it is widely used by both students and professional software developers.

As a student, I have found that AI is helpful for learning programming concepts and completing coding tasks more quickly. It can explain difficult topics, suggest solutions, and help identify errors in my code. However, I also understand that AI should be used as a supporting tool, while the final decisions and code verification should always be done by the developer.

The following are some of the main advantages of AI in software development.

- **Increased Productivity**

AI helps developers write code faster by providing code suggestions, generating functions, and automating repetitive programming tasks. This reduces development time and improves efficiency.

- **Faster Debugging**

AI can identify syntax errors, logical mistakes, and possible bugs quickly. It also suggests possible solutions, making the debugging process faster and easier.

- **Improved Code Quality**

AI recommends better coding practices, cleaner code structures, and more meaningful variable names. This makes the code easier to read, understand, and maintain.

- **Automated Testing**

AI can generate test cases and unit tests automatically. This helps developers detect errors early and improves the reliability of software applications.

- **Better Learning Support**

AI explains programming concepts, provides sample code, and helps beginners understand error messages. This makes learning programming easier and more interactive.

- **Time Saving**

By automating repetitive tasks such as code generation, documentation, and testing, AI allows developers to complete software projects more quickly.

- **Better Team Collaboration**

AI can generate documentation, summarize code changes, and explain existing code. This helps team members understand projects more easily and improves collaboration.

- **Increased Software Reliability**

AI assists in detecting bugs and improving code quality before software is released, helping developers build more reliable and stable applications.

1.4 Risks and limitations

Although AI-assisted programming has become a valuable tool in software development, it also has several risks and limitations. AI can help developers write code, detect errors, and complete programming tasks more quickly. However, AI is not always accurate and cannot fully understand the requirements of every project. Sometimes it generates incorrect solutions or provides suggestions that are not suitable for the given problem. Therefore, developers should always review, test, and verify AI-generated code before using it in real-world applications.

As a student, I have found that AI is very useful for learning programming concepts and solving coding problems. However, I have also noticed that some AI-generated answers contain mistakes or are not the best solution. If I use the code without understanding it, I may

struggle to fix errors or make changes later. Because of this, AI should be used as a learning assistant rather than as a replacement for programming knowledge and problem-solving skills.

The following are some of the main risks and limitations of AI-assisted programming.

- **Incorrect or Inaccurate Code**

AI may generate code that contains syntax errors, logical mistakes, or incomplete solutions. Developers should always test and verify the generated code before using it.

- **Security Risks**

AI-generated code may include security vulnerabilities such as weak authentication, poor input validation, or unsafe coding practices. These issues should be carefully reviewed before deployment.

- **Overdependence on AI**

Relying too much on AI can reduce a programmer's ability to think independently and solve problems. Developers should continue improving their own programming skills.

- **Limited Understanding of Requirements**

AI cannot always understand the complete context or business requirements of a project. As a result, the generated code may not fully meet the user's needs.

- **Privacy and Data Security**

Using AI tools with confidential source code or sensitive information may create privacy risks, especially when cloud-based AI services are used.

- **Outdated or Inefficient Solutions**

AI may sometimes recommend older programming methods or inefficient algorithms instead of the latest best practices.

- **Need for Human Verification**

AI-generated code should always be reviewed, tested, and validated by developers to ensure it is accurate, secure, and suitable for the project.

- **Lack of Human Creativity**

AI can assist with coding tasks, but it cannot replace human creativity, critical thinking, and decision-making that are required for designing and developing high-quality software.

Task 02

2.1

I selected inventory management system

An Inventory Management System is a software application used to manage and organize product information in a business or retail store. It helps users keep track of available products, their prices, quantities, and categories in an efficient manner. Instead of maintaining inventory records manually, the system stores product information digitally, making it easier to update, search, and monitor inventory.

For this project, a Java console-based Inventory Management System was developed. The system allows users to add new products, update existing product details, delete products, search for products by their ID, display all available products, update stock quantities, identify low-stock items, and calculate the total value of the inventory. These features help users manage inventory more effectively while reducing manual work.

The main objective of this project is to develop the system in two different ways. The first version is developed using the traditional programming approach without AI assistance. The second version is developed with the support of AI-assisted programming tools. By comparing both versions, it is possible to evaluate how AI can improve software development in terms of development time, code quality, productivity, and overall programming experience.

2.2 Student Management System Code

Version A – Basic java Function and OOP concept without AI

```

1  public class Product {
2      private int productId;
3      private String productName;
4      private String category;
5      private double price;
6      private int quantity;
7
8      public Product(int productId, String productName, String category, double price, int quantity) {
9          this.productId = productId;
10         this.productName = productName;
11         this.category = category;
12         this.price = price;
13         this.quantity = quantity;
14     }
15
16     public int getProductId() { return productId; }
17
18     public String getProductName() { return productName; }
19
20     public String getCategory() { return category; }
21
22     public double getPrice() { return price; }
23
24     public int getQuantity() { return quantity; }
25
26     public void setProductName(String productName) { this.productName = productName; }
  
```

```

Inventory.java x Main.java Product.java
1 import java.util.ArrayList;
2
3 public class Inventory { no usages
4
5     private ArrayList<Product> products = new ArrayList<>(); 8 usages
6
7     public void addProduct(Product product) { no usages
8         for (Product p : products) {
9             if (p.getProductID() == product.getProductID()) {
10                 System.out.println("Product ID already exists.");
11                 return;
12             }
13         }
14
15         products.add(product);
16         System.out.println("Product added successfully.");
17     }
18
19     public void viewProducts() { no usages
20         if (products.isEmpty()) {
21             System.out.println("No products available.");
22             return;
23         }
24
25         for (Product p : products) {
26             System.out.println(p);

```

```

Inventory.java Main.java x Product.java
1 import java.util.Scanner;
2
3 public class Main { no usages
4
5     public static void main(String[] args) { no usages
6
7         Scanner sc = new Scanner(System.in);
8         Inventory inventory = new Inventory();
9         int choice;
10
11         do {
12             System.out.println("\n==== Inventory Management System =====");
13             System.out.println("1. Add Product");
14             System.out.println("2. View All Products");
15             System.out.println("3. Search Product");
16             System.out.println("4. Update Product");
17             System.out.println("5. Delete Product");
18             System.out.println("6. Update Stock");
19             System.out.println("7. Low Stock Report");
20             System.out.println("8. Calculate Inventory Value");
21             System.out.println("9. Exit");
22             System.out.print("Enter your choice: ");
23
24             choice = sc.nextInt();
25
26             switch (choice) {

```

Version B – Using AI (add Error handling,use advance concept)

```

1  /**
2   * Product.java
3   * -----
4   * Represents a single product in the Inventory Management System.
5   * This class stores product details and provides methods
6   * to access, modify, and display product information.
7   */
8
9  public class Product { 17 usages
10
11     private int productId; 5 usages
12     private String productName; 5 usages
13     private String category; 5 usages
14     private double price; 6 usages
15     private int quantity; 8 usages
16
17     /**
18      * Default constructor.
19      */
20     public Product() { no usages
21     }
22
23     /**
24      * Parameterized constructor.

```

```

1  import java.util.ArrayList;
2
3  public class Inventory { 2 usages
4
5     private ArrayList<Product> products = new ArrayList<>(); 11 usages
6
7     /**
8      * Adds a new product after checking for duplicate IDs.
9      */
10     public boolean addProduct(Product product) { 1 usage
11
12         if (findProductById(product.getProductId()) != null) {
13             System.out.println("Product ID already exists.");
14             return false;
15         }
16
17         products.add(product);
18         System.out.println("Product added successfully.");
19         return true;
20     }
21
22     /**
23      * Displays all products.
24     */

```

```

1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  public class Main {
5
6     public static void main(String[] args) {
7
8         Scanner input = new Scanner(System.in);
9         Inventory inventory = new Inventory();
10
11         int choice;
12
13         do {
14
15             System.out.println("\n=====");
16             System.out.println("                INVENTORY MANAGEMENT SYSTEM");
17             System.out.println("=====");
18             System.out.println("1. Add Product");
19             System.out.println("2. View All Products");
20             System.out.println("3. Search Product by ID");
21             System.out.println("4. Search Product by Name");
22             System.out.println("5. Update Product");
23             System.out.println("6. Delete Product");
24             System.out.println("7. Update Stock");

```

2.3 Outputs

Version A

```

Terminal Local x + v
symbol: class Product
location: class Main
5 errors
error: compilation failed

PS C:\Users\DELL\Desktop\IT Report> javac Main.java Product.java Inventory.java

PS C:\Users\DELL\Desktop\IT Report> java Main
\n===== Inventory Management System =====
1. Add Product
2. View All Products
3. Search Product
4. Update Product
5. Delete Product
6. Update Stock
7. Low Stock Report
8. Calculate Inventory Value
9. Exit
Enter your choice:

```

Version B

```

Terminal Local x + v
Enter Product ID: 201
New Product Name: ABCD
New Category: XYZ
New Price: 650
New Quantity: 80
Product not found.

=====
INVENTORY MANAGEMENT SYSTEM
=====
1. Add Product
2. View All Products
3. Search Product by ID
4. Search Product by Name
5. Update Product
6. Delete Product
7. Update Stock
8. Low Stock Report
9. Inventory Summary
10. Calculate Inventory Value
11. Exit
Enter your choice:

Terminal Local x + v
Product not found.

=====
INVENTORY MANAGEMENT SYSTEM
=====
1. Add Product
2. View All Products
3. Search Product by ID
4. Search Product by Name
5. Update Product
6. Delete Product
7. Update Stock
8. Low Stock Report
9. Inventory Summary
10. Calculate Inventory Value
11. Exit
Enter your choice: 11

Thank you for using the Inventory Management System.
Program terminated successfully.

PS C:\Users\DELL\Desktop\AI codes>

```

2.4 Comparison Between Version A and Version B

The Inventory Management System was developed using two different approaches: the traditional programming approach and the AI-assisted programming approach. Both versions provide the same core functionality, but the development process and code quality differ in several aspects.

Development Time

The traditional version required more time because every class, method, and logic had to be written manually. Considerable time was also spent debugging syntax errors and testing the program. In contrast, the AI-assisted version was completed more quickly because AI tools provided code suggestions, generated methods, and helped solve programming issues. This reduced the overall development time significantly.

Code Quality

The traditional version contains simple and straightforward code that is suitable for beginners. However, the code structure is basic and includes fewer validations and helper methods. The AI-assisted version has a cleaner structure with better formatting, modular methods, improved naming conventions, and detailed comments. It also includes additional features such as stock status, inventory summary, and search by product name, making the code more organized and professional.

Error Handling

In the traditional version, only basic validation was implemented, such as checking for duplicate product IDs. Invalid user input could cause the program to terminate if not entered correctly. The AI-assisted version provides stronger error handling by using input validation and exception handling (try-catch) to prevent the program from crashing due to incorrect user input. It also displays meaningful error messages to guide the user.

Readability and Maintainability

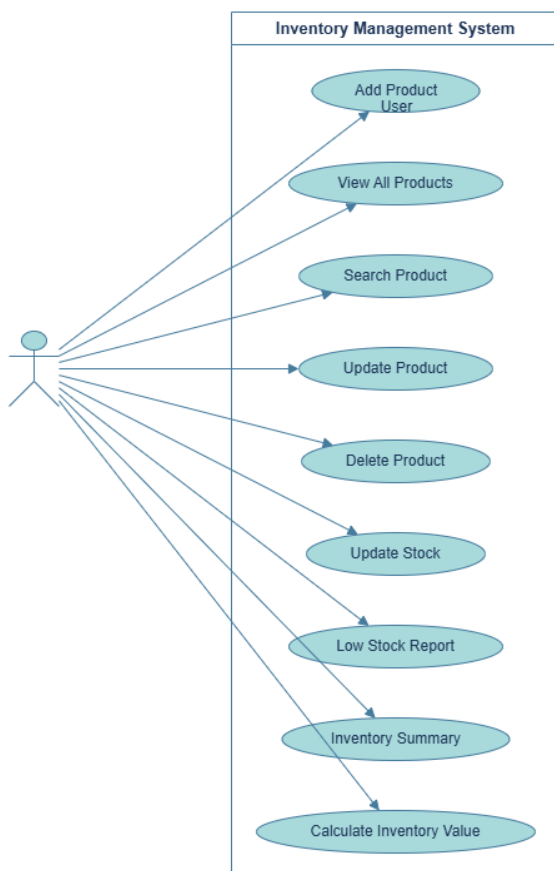
The traditional version is easy to understand but becomes harder to maintain as the project grows because most logic is written in simple methods with minimal documentation. The AI-assisted version is more readable due to proper formatting, descriptive method names, Javadoc comments, and modular design. These improvements make the code easier to understand, modify, and maintain in the future.

Summary

Aspect	Version A – Traditional	Version B – AI-Assisted
Development Time	Longer because all code was written manually	Shorter due to AI-generated suggestions and faster implementation
Code Quality	Basic structure with simple programming techniques	Cleaner, modular, and better organized with additional features
Error Handling	Basic validation with limited error checking	Better validation, exception handling, and informative error messages
Readability & Maintainability	Easy to understand but less organized	Well-structured, documented, and easier to maintain

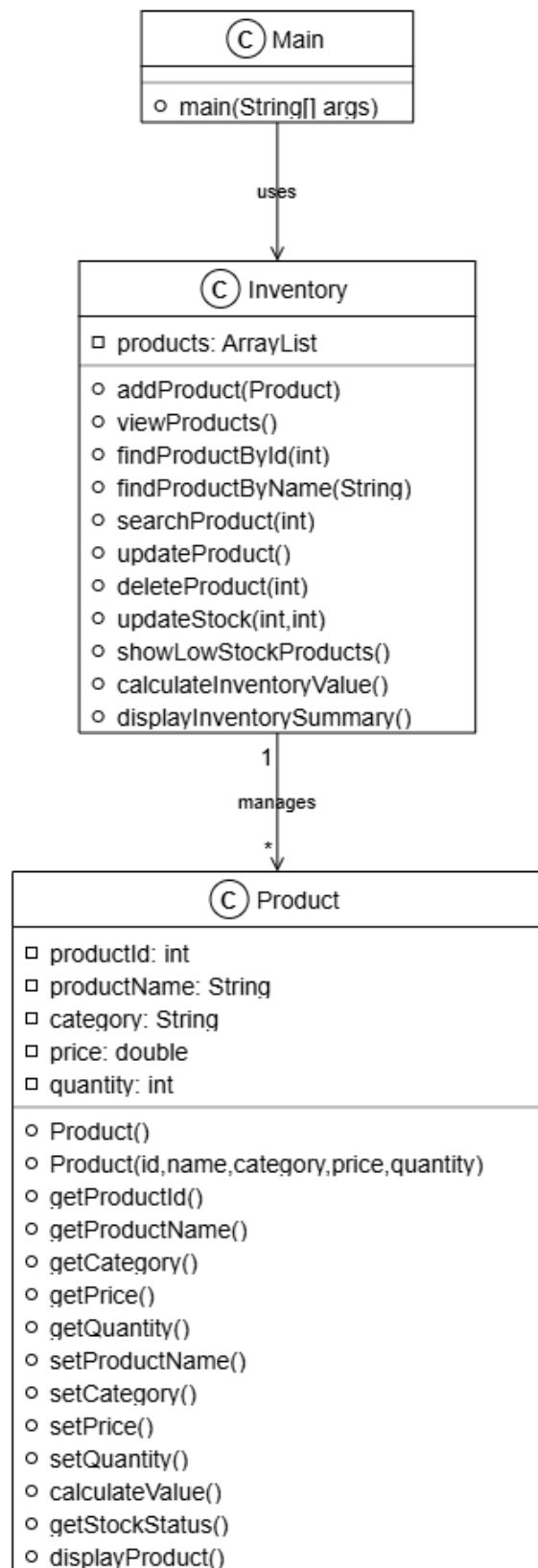
2.5 UML Diagrams

User case diagram

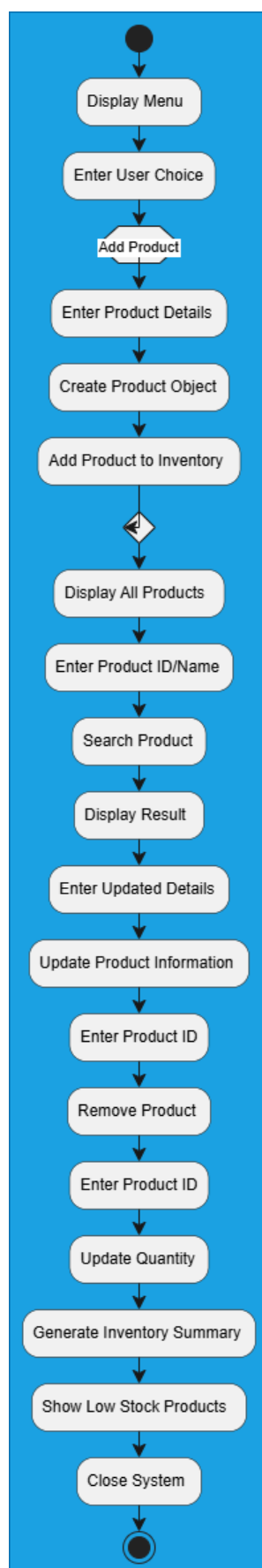


Class Diagram

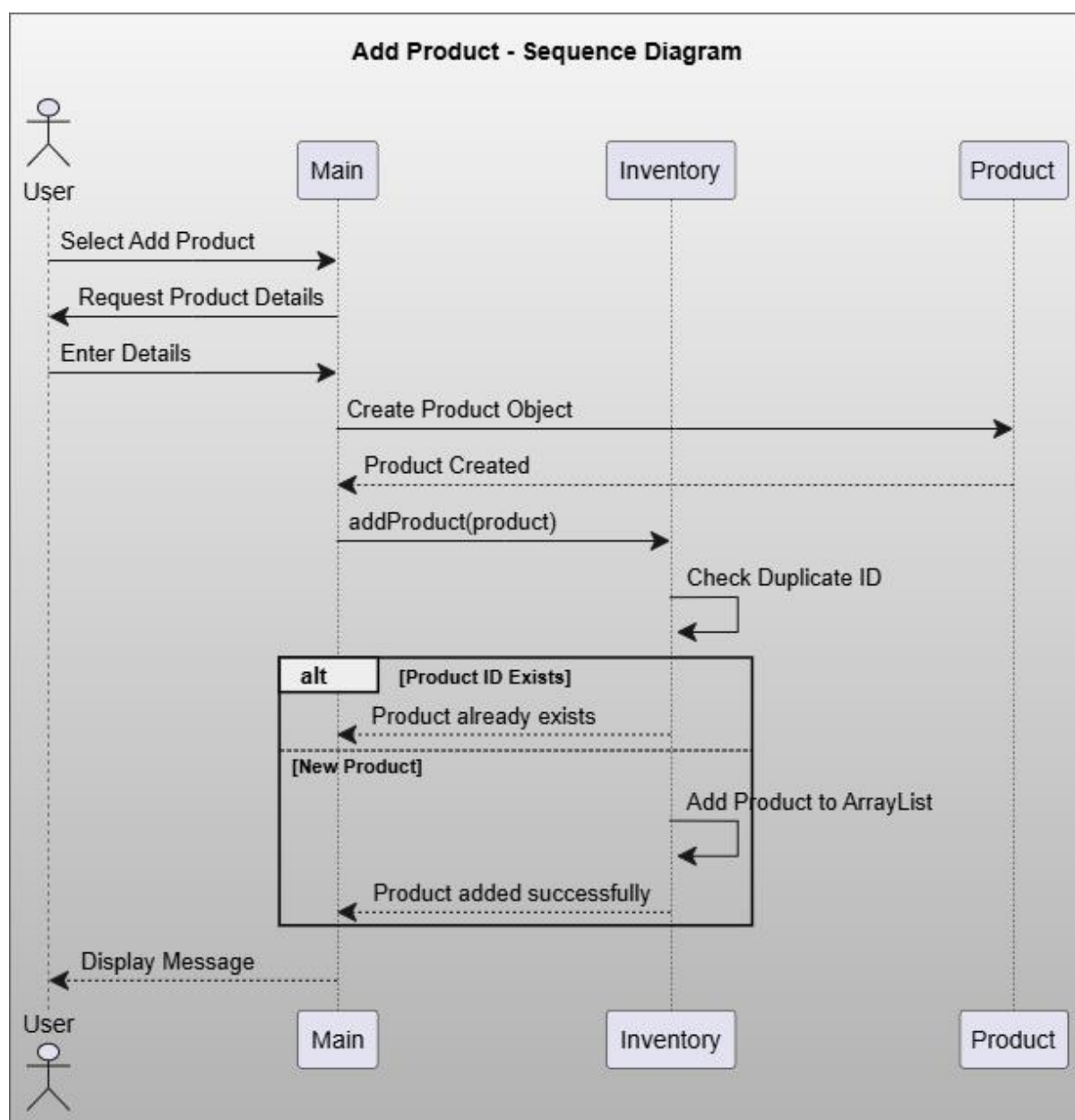
Inventory Management System - Class Diagram



Activity Diagram



Sequence Diagram



Task 03

Critical Analysis

How AI changed my coding approach

AI changed my coding approach by helping me understand the problem and plan the system before writing code. It helped me design the structure of the Inventory Management System by identifying classes such as Main, Inventory, and Product. AI also helped me find errors quickly, improve code quality, and learn better programming practices. As a result, I was able to complete coding tasks more efficiently.

Whether AI improved or reduced my understanding

AI improved my understanding by providing explanations and examples for difficult programming concepts. It helped me understand object-oriented programming concepts such as classes, objects, methods, and relationships between classes. However, using AI without understanding the generated code can reduce learning. Therefore, AI should be used as a learning tool while making sure that the programmer understands the logic behind the code.

In which AI should NOT be used

AI should not be used to completely replace a developer's thinking and problem-solving ability. Students should avoid copying AI-generated code without understanding it. AI-generated solutions should not be directly applied to important systems such as banking, healthcare, or security applications without proper testing and review. It should also not be used as a shortcut to avoid learning programming concepts.

Ethical considerations in AI generated code

When using AI-generated code, developers should take responsibility for the final output. The generated code should be reviewed, tested, and modified according to the project requirements. Developers should consider issues such as plagiarism, copyright, privacy, and security when using AI tools. AI should be used responsibly while keeping human judgement and decision-making as the main part of software development.

Task 04 - Conclusion

Final opinion on AI in programming education and industry

AI has become an important technology that is changing the way programming is learned and practiced. In programming education, AI provides students with quick access to explanations, examples, and guidance when learning difficult concepts. It can help beginners understand programming logic, identify errors, and explore different approaches to solving problems. However, students should not depend completely on AI because developing problem-solving skills, logical thinking, and programming experience requires personal practice and effort. AI should be used as a supportive learning tool that improves understanding rather than a method of simply obtaining answers.

In the software industry, AI has improved the efficiency of developers by helping with tasks such as code generation, debugging, software testing, documentation, and improving existing code. It allows developers to complete tasks faster and focus more on creative problem-solving and system design. However, AI-generated code may sometimes contain errors, security issues, or unsuitable solutions, so professional developers must review and test the output before using it in real-world applications.

Overall, AI has a positive impact on programming education and the software industry when it is used responsibly. It can increase productivity, support learning, and help developers create better software. However, human knowledge, creativity, ethical awareness, and decision-making remain essential. The future of programming is not about replacing programmers with AI, but about programmers using AI effectively as a powerful tool to improve their abilities and work quality.

References

- **Oracle. (2024).** *Java Documentation*. <https://docs.oracle.com/en/java/>
- Google. (2024). Gemini for Developers. <https://ai.google.dev>
- Draw.io. (2024). Diagrams.net Documentation. <https://www.diagrams.net/>
- OpenAI. (2024). ChatGPT. <https://chatgpt.com>
- Anthropic. (2024). Claude AI. <https://claude.ai/>

GIT HUB Link

https://github.com/hirushan900953/it_assesment